# Information Retrieval Indexing

## Lecture 2: Posting Lists, Compression & ANN Indexing

Avishek Anand

Information Retrieval

February 9, 2026

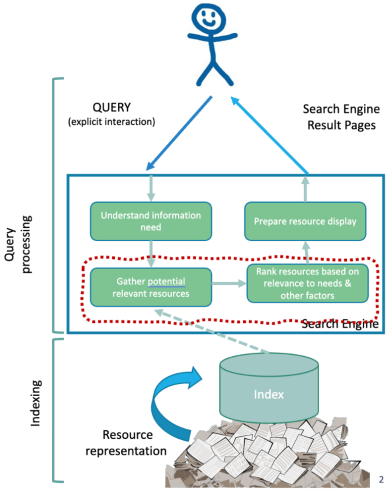
# Outline


1. The Information Retrieval Process
2. Inverted Index Posting Lists
3. Query Processing Algorithms
4. Index Compression
5. Approximate k-NN Indexing for Dense Retrieval
6. Summary

# Course Overview

1. Introduction to IR: VSM & Boolean Retrieval ✓
2. **Indexing: Sparse and Dense** ← Today
3. Query Understanding & Expansion
4. Evaluation of IR Systems
5. Ranking: BM25, Language Models, and Beyond
6. Embeddings & Neural Ranking
7. Dense Retrieval
8. Learning to Rank

# The Complete IR Pipeline



# Text Preprocessing & Indexing

## The Foundation of Search

Building an IR system happens in three main stages:

- **Stage 1: Offline Indexing**
  - Documents are preprocessed (tokenized, stemmed, etc.)
  - Inverted indexes are built and compressed
- **Stage 2: Online Query Processing**
  - Query is preprocessed **identically** to documents
  - Match terms in the index
- **Stage 3: Ranking**
  - Scored according to a model (TF-IDF, BM25, etc.)

# Step-by-Step: Query Processing

**Applying consistent preprocessing**

**Example**

**User Query:** "Running Dogs in the Park"

1. **Tokenization:** [Running, Dogs, in, the, Park]
2. **Lowercasing:** [running, dogs, in, the, park]
3. **Stop word removal:** [running, dogs, park]
4. **Stemming:** [run, dog, park]

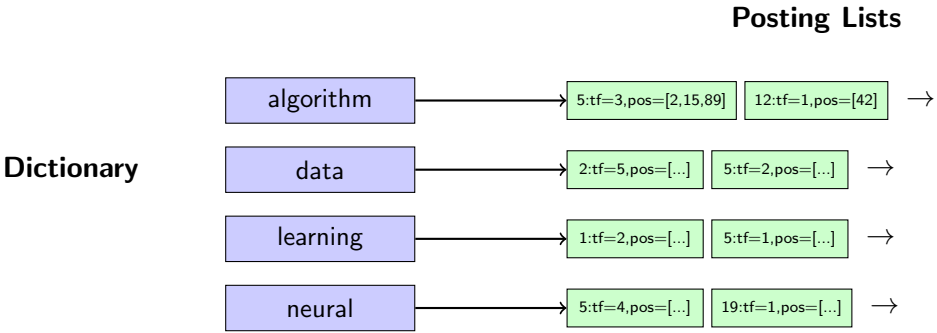
**Processed Query:** [run, dog, park]

**The Consistency Principle**

**Preprocessing must be identical for indexing and retrieval!**

# Inverted Index: Quick Recap

**Core data structure for text search**




**Each posting contains:**

- Document ID

# Posting List Formats (1/2)

## Basic Format: DocID Only

$$\langle \text{docID}_1, \text{docID}_2, \text{docID}_3, \ldots \rangle$$

- Minimal information
- Boolean queries only
- Example: [5, 12, 27, 89, 102]

## With Frequencies

$$\langle (\text{docID}_1, \text{tf}_1), (\text{docID}_2, \text{tf}_2), \ldots \rangle$$

- Enables TF-IDF, BM25 scoring
- Example: [(5, 3), (12, 1), (27, 2), (89, 1), (102, 5)]

# Posting List Formats (2/2)

**With Positions (Full Inverted Index)**

$$\langle(\text{docID}_1, \text{tf}_1, [\text{pos}_{1,1}, \text{pos}_{1,2}, \ldots]), \ldots\rangle$$

- Enables phrase queries, proximity scoring
- Example: [(5, 3, [2, 15, 89]), (12, 1, [42]), ...]
- Larger storage, but most flexible

**Recommendation:** Use positions for production systems (enables phrase queries)

# Posting List Properties

## Key Properties

1. **Sorted by DocID**
   - Enables efficient merge operations
   - Required for compression (gap encoding)
   - Allows skip pointers
2. **Variable Length**
   - Rare terms: short lists (e.g., "syzygy": 10 docs)
   - Common terms: long lists (e.g., "the": millions of docs)
   - Power-law distribution (Zipf's law)
3. **Stored Sequentially**
   - Good cache locality
   - Sequential disk reads (fast)
   - Compressed for storage efficiency

# Skip Lists: Motivation

## Problem

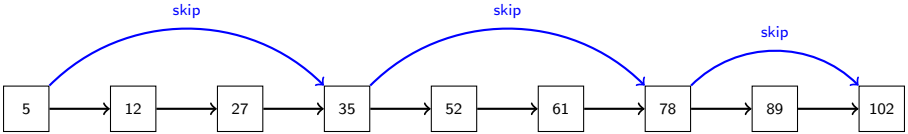
Merging long posting lists can be slow

- Query: "algorithm AND learning"
- "algorithm": 1M documents
- "learning": 500K documents
- Naive merge: scan both lists completely

## Example: Merging Without Skip Lists

**List A:** [1, 5, 10, 15, 20, 25, 30, ...] (1M entries)
**List B:** [3, 6, 9, 12, 15, 18, 21, ...] (500K entries)
Finding intersection requires comparing **every** element:
Compare 1 vs 3, 5 vs 3, 5 vs 6, 10 vs 6, 10 vs 9, ...

# Skip Lists: Structure




**Posting List with Skip Pointers**

**Skip pointers properties:**

- Placed at regular intervals (e.g., every $\sqrt{n}$ elements)
- Point to future positions in the list
- Store the maximum docID they skip over

**Trade-off:**

- ✓ Faster intersection (skip irrelevant parts)
- ✗ Extra storage for pointers

# Skip Lists: Intersection Algorithm

```
Input: Two posting lists p_1, p_2 with skip pointers
Output: Intersection of p_1 and p_2

i_1 ← 0, i_2 ← 0 {Pointers to current positions}
result ← []
while i_1 < |p_1| AND i_2 < |p_2| do
  if p_1[i_1].docID = p_2[i_2].docID then
    Append p_1[i_1].docID to result
    i_1 ← i_1 + 1, i_2 ← i_2 + 1
  else if p_1[i_1].docID < p_2[i_2].docID then
    if p_1[i_1] has skip AND p_1[i_1].skip.maxDocID ≤ p_2[i_2].docID then
      i_1 ← p_1[i_1].skip {Use skip pointer}
    else
      i_1 ← i_1 + 1 {Advance normally}
    end if
  else
    if p_2[i_2] has skip AND p_2[i_2].skip.maxDocID ≤ p_1[i_1].docID then
      i_2 ← p_2[i_2].skip
    else
      i_2 ← i_2 + 1
    end if
  end if
end while
return result
```

# Skip Lists: Example Intersection

**Query:** Find intersection of two lists

**List A:** [5, 12, 27, 35, 52, 61, 78, 89, 102] (skip every 3)
**List B:** [12, 35, 52, 89, 102, 150]

**Trace:**

1. Compare 5 vs 12 → 5 ¡ 12, skip from 5 to 35
2. Compare 35 vs 12 → 35 ¿ 12, advance B to 35
3. Compare 35 vs 35 → **Match!** → Output 35
4. Compare 52 vs 52 → **Match!** → Output 52
5. Compare 61 vs 89 → 61 ¡ 89, skip from 61 to 89
6. Compare 89 vs 89 → **Match!** → Output 89
7. Compare 102 vs 102 → **Match!** → Output 102
8. List A exhausted

**Result:** [35, 52, 89, 102]

# Skip List Placement Strategy

**How many skip pointers?**

Trade-off between space and speedup

**Common strategies:**

1. $\sqrt{n}$ **spacing** (optimal for random access)
   - For list of length $n$, place $\sqrt{n}$ skip pointers
   - Each skip jumps $\sqrt{n}$ positions
   - Example: 10,000 elements $\rightarrow$ 100 skips, each spanning 100 elements
2. **Fixed interval** (simpler)
   - Skip every $k$ elements (e.g., $k = 10$)
   - Easier to implement and maintain
3. **Multi-level skip lists**
   - Skip pointers at multiple granularities
   - Level 1: every 10, Level 2: every 100, etc.
   - More complex but faster for very long lists

# Query Processing: The Task

## Input

- Query: set of terms $q = \{t_1, t_2, \ldots, t_k\}$
- Inverted index with posting lists
- Scoring function (e.g., BM25, cosine similarity)

## Output

- Ranked list of top-$N$ documents by score
- Typically $N = 10$ or $N = 1000$

**Key challenge:** Efficiency

- Can't score all documents (millions/billions)
- Must process posting lists cleverly
- Different algorithms for different scenarios

# Two Main Paradigms

## Document-at-a-Time (DAAT)

- Process one document completely
- Compute full score for that doc
- Move to next document
- Keep top-$k$ heap

**Analogy:** Grading exams

- Grade all questions for student 1
- Then all questions for student 2
- Etc.

## Term-at-a-Time (TAAT)

- Process one query term completely
- Update partial scores for all docs
- Move to next query term
- Accumulate scores

**Analogy:** Grading exams

- Grade question 1 for all students
- Then question 2 for all students
- Etc.

## Which is better?

Depends on: query length, list lengths, scoring function, hardware

# Document-at-a-Time (DAAT)

```
Input: Query terms {t_1, t_2, ..., t_k}, posting lists
Output: Top-N documents

Fetch posting lists for all query terms
heap ← empty min-heap of size N {Store top-N docs}
nextDocID ← minimum docID across all posting lists
while posting lists not exhausted do
    d ← nextDocID
    score ← 0
    for each term t_i in query do
        if t_i's posting list contains d then
            score ← score + termScore(t_i, d) {BM25, TF-IDF, etc.}
            Advance t_i's pointer
        end if
    end for
```

# DAAT Example

**Query:** "neural learning"
**Posting lists:**

- neural: [5, 12, 27, 89]
- learning: [5, 27, 35, 89]

**Processing (top-2):**

1. **Doc 5:** Both terms present $\rightarrow$ score $=$ $\text{BM25}(\text{neural}, 5) + \text{BM25}(\text{learning}, 5) = 2.3$
   - Heap: [(5, 2.3)]
2. **Doc 12:** Only "neural" $\rightarrow$ score $= \text{BM25}(\text{neural}, 12) = 1.1$
   - Heap: [(5, 2.3), (12, 1.1)]
3. **Doc 27:** Both terms $\rightarrow$ score $= 3.1$
   - Heap: [(5, 2.3), (27, 3.1)] (evict doc 12)
4. **Doc 35:** Only "learning" $\rightarrow$ score $= 0.9$
   - Heap unchanged (score too low)
5. **Doc 89:** Both terms $\rightarrow$ score $= 2.5$

# DAAT with Skip Pointers

**Optimization:** Use skip pointers to jump to next candidate document

```
Find document d that appears in at least one posting list
if d appears in all posting lists then
    Compute full score for d
else if d appears in some but not all lists then
    Compute partial score for d (only from matching terms)
end if
Use skip pointers to advance to next candidate efficiently
```

**Key insight:**

- Shortest posting list determines candidates
- Use skips to align longer lists to candidates
- Much faster when lists have very different lengths

# Term-at-a-Time (TAAT)

```
Input: Query terms {t_1, t_2, ..., t_k}, posting lists
Output: Top-N documents

scores ← empty hash table (docID → score)
for each query term t_i do
    Fetch posting list for t_i
    for each document d in posting list do
        if d not in scores then
            scores[d] ← 0
        end if
        scores[d] ← scores[d] + termScore(t_i, d)
    end for
end for
Sort documents by score (descending)
return top-N documents
```

# TAAT Example

**Query:** "neural learning"
**Posting lists:**

- neural: [5, 12, 27, 89]
- learning: [5, 27, 35, 89]

**Processing:**
**Step 1: Process "neural"**

- Doc 5: scores[5] $= 1.5$
- Doc 12: scores[12] $= 1.1$
- Doc 27: scores[27] $= 1.6$
- Doc 89: scores[89] $= 1.3$

**Step 2: Process "learning"**

- Doc 5: scores[5] $= 1.5 + 0.8 = 2.3$
- Doc 27: scores[27] $= 1.6 + 1.5 = 3.1$
- Doc 35: scores[35] $= 0.9$

# DAAT vs. TAAT Comparison

| Aspect | DAAT | TAAT |
|---|---|---|
| Memory | Low (only top-$k$ heap) | High (all candidate scores) |
| Cache locality | Good (sequential per doc) | Poor (random access) |
| Early termination | Easy (threshold pruning) | Hard (need all terms) |
| Parallelization | Harder | Easier (independent terms) |
| Skip pointers | Very effective | Less effective |
| Long queries | Better | Worse (more memory) |
| Conjunctive queries | Natural | Must filter at end |
| **Modern systems** | **Preferred** | Used in batch scoring |

## Practical Choice

- **DAAT:** Most production systems (Lucene, Elasticsearch, Solr)
- **TAAT:** Batch re-ranking, research systems

# Query Optimization: MaxScore

**Observation**

Not all query terms contribute equally to final scores

**MaxScore Algorithm:**

1. Compute maximum possible contribution for each term

$$\text{maxScore}(t) = \max_{d} \text{score}(t, d)$$

2. Sort query terms by maxScore (descending)
3. Maintain threshold $\theta$ = score of $k$-th best document so far
4. Skip processing term $t$ for document $d$ if:

$$\text{score}(d) + \text{maxScore}(t) < \theta$$

(Can't possibly make top-$k$)

**Result:** Avoid scoring low-impact terms for documents that won't reach top-$k$

# MaxScore Example

**Query:** "neural network learning"

**Max scores:**

- neural: maxScore $= 2.5$
- network: maxScore $= 2.0$
- learning: maxScore $= 1.5$

**Sorted by maxScore:** [neural, network, learning]

**Processing doc 42 (top-10 query):**

1. Current 10th-best score: $\theta = 5.0$
2. Score so far for doc 42: 3.0 (from "neural" and "network")
3. Max additional score: 1.5 (from "learning")
4. Total possible: $3.0 + 1.5 = 4.5 < 5.0$
5. **Skip** scoring "learning" for doc 42 (can't reach top-10)

# Query Optimization: WAND

## Weak AND (WAND)

Advanced optimization combining DAAT with dynamic pruning

**Key ideas:**

1. Maintain upper bounds on term contributions
2. Process posting lists in parallel (like DAAT)
3. Skip entire documents if upper bound ¡ threshold
4. Dynamically update threshold as better docs found

**WAND pivot algorithm:**

- Find "pivot" document where sum of upper bounds $\geq \theta$
- All documents before pivot can be safely skipped
- Use skip pointers to jump to pivot

# Impact-Ordered Indexes

**Alternative Index Organization**

Instead of sorting postings by docID, sort by impact (score)

**Traditional (docID-ordered):**

- neural: [(5, tf=1), (12, tf=3), (27, tf=1), (89, tf=5)]
- Sorted by docID

**Impact-ordered:**

- neural: [(89, tf=5), (12, tf=3), (5, tf=1), (27, tf=1)]
- Sorted by term frequency (or TF-IDF score)

**Benefits:**

- **Early termination:** Stop after processing top-$k$ high-impact postings
- **Safe to skip:** Low-impact docs unlikely to reach top-$k$
- **Query-independent:** Pre-computed at index time

# Why Compress Indexes?

## Motivation

1. **Storage costs**
   - Uncompressed index: 10-20% of raw collection size
   - For 100TB collection: 10-20TB index
   - Compression: 5-10× reduction → 1-4TB
2. **I/O efficiency**
   - Disk/SSD bandwidth: 500 MB/s
   - Decompression: 2-5 GB/s
   - Reading compressed data is **faster**!
3. **Cache efficiency**
   - More index fits in RAM
   - Better cache hit rates
   - Faster query processing

# What to Compress?

## Dictionary

**Size:** 100-500 MB (small)
**Techniques:**

- Front coding
- Suffix arrays
- Finite-state automata

**Impact:** Minor (already small)

## Postings Lists

**Size:** Gigabytes to terabytes
**Techniques:**

- Gap encoding
- Variable-byte
- Gamma/Delta codes
- PForDelta

**Impact:** **Major** (most of index)

## Typical Index Breakdown

- Dictionary: 2%
- Postings (docIDs): 70%

# Dictionary Compression: Front Coding

**Observation**

Sorted dictionary has many shared prefixes

**Uncompressed:**

```
algorithm
algorithmic
algorithmically
algorithms
alien
align
alignment
```

**Storage:** 64 bytes

**Front coded:**

```
9 algorithm
  2 ic
  5 ally
  1 s
5 ali
  2 en
  2 gn
  5 ment
```

**Storage:** $\approx$ 38 bytes

# Postings Compression: Gap Encoding

**Key Insight**

DocIDs in postings are sorted $\rightarrow$ store gaps instead of absolute values

**Original postings (docIDs):**

$$[5, 127, 214, 215, 512, 1024, 2048, 10239]$$

**Gaps (d-gaps):**

$$[5, 122, 87, 1, 297, 512, 1024, 8191]$$

**Why This Helps**

- **Original:** Numbers range from 5 to 10,239 (need 14 bits each)
- **Gaps:** Most numbers small (need fewer bits)
- Small numbers compress much better!

# Gap Distribution

**Zipf's Law**

Term frequencies follow power-law distribution

**Implications for gaps:**

- **Rare terms:** Large gaps (appear infrequently)
- **Common terms:** Small gaps (appear often)
- **Overall:** Most gaps are small!

**Example**

**Rare term "syzygy":** [15, 5023, 89234]
Gaps: [15, 5008, 84211] (large gaps)

**Common term "the":** [1, 2, 5, 7, 8, 10, 12, ...]
Gaps: [1, 1, 3, 2, 1, 2, 2, ...] (small gaps)

# Variable Byte (VB) Encoding

## Core Idea

Use variable number of bytes to encode integers

- Small numbers: 1 byte
- Larger numbers: 2+ bytes
- Self-delimiting: continuation bit indicates if more bytes follow

**Encoding scheme:**

- Use 7 bits per byte for data
- Use 1 bit (MSB) as continuation flag
- 1 = last byte, 0 = more bytes follow

| Number | Binary | VB Encoding |
|---|---|---|
| 5 | 101 | 1_0000101 (1 byte) |

# VB Encoding: Detailed Examples

**Example 1: Encode 5**

1. Binary: 101 (3 bits)
2. Fits in 7 bits → use 1 byte
3. Data: 0000101, continuation: 1 (last byte)
4. Result: 10000101

**Example 2: Encode 824**

1. Binary: 1100111000 (10 bits)
2. Needs 2 bytes (10 ¿ 7)
3. Split into 7-bit chunks (from right): 0111000, 0000110
4. First byte (higher bits): 0_0000110 (more bytes follow)
5. Second byte (lower bits): 1_0111000 (last byte)
6. Result: 00000110 10111000

**Decoding:**

# VB: Compressing a Posting List

**Original docIDs:** [5, 127, 214, 215]

**Step 1: Compute gaps**

- Gap 1: 5 - 0 = 5
- Gap 2: 127 - 5 = 122
- Gap 3: 214 - 127 = 87
- Gap 4: 215 - 214 = 1

Gaps: [5, 122, 87, 1]

**Step 2: VB encode each gap**

- 5 → 10000101 (1 byte)
- 122 → 11111010 (1 byte)
- 87 → 11010111 (1 byte)
- 1 → 10000001 (1 byte)

**Result:** 4 bytes compressed vs. 16 bytes uncompressed (32-bit ints)

# Gamma Encoding

## Motivation

VB uses 8 bits to encode 7 bits (12.5% overhead)
Can we do better for very small numbers?

**Gamma encoding:**

1. Let $N = \lfloor \log_2 x \rfloor$ (position of highest bit)
2. Encode $N$ in unary: $N$ zeros + one 1
3. Encode offset $x - 2^N$ in $N$ bits

## Examples

**Encode 5:**

- Binary: 101, $N = 2$ (highest bit at position 2)
- Unary for $N = 2$: 001 (two zeros + 1)

# Gamma Encoding: More Examples

| $x$ | **Binary** | $N$ | **Offset** | **Gamma Code** |
|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 1 (1 bit) |
| 2 | 10 | 1 | 0 | 010 (3 bits) |
| 3 | 11 | 1 | 1 | 011 (3 bits) |
| 4 | 100 | 2 | 0 | 00100 (5 bits) |
| 5 | 101 | 2 | 1 | 00101 (5 bits) |
| 9 | 1001 | 3 | 1 | 0001001 (7 bits) |
| 100 | 1100100 | 6 | 36 | 000000_1100100 (13 bits) |

**Observations:**

- Very efficient for small numbers (1-15)
- Encoding length: $2\lfloor \log_2 x \rfloor + 1$ bits
- Self-delimiting (no need for length prefix)
- Optimal for geometric distributions

# VB vs. Gamma Comparison

| Number | VB (bits) | Gamma (bits) | Better |
|---|---|---|---|
| 1 | 8 | 1 | Gamma |
| 5 | 8 | 5 | Gamma |
| 10 | 8 | 7 | Gamma |
| 100 | 8 | 13 | VB |
| 127 | 8 | 13 | VB |
| 128 | 16 | 15 | Gamma |
| 1000 | 16 | 19 | VB |
| 10000 | 16 | 27 | VB |

## Trade-offs

- **Gamma:** Better compression for numbers ¡ 128
- **VB:** Simpler, faster, better for larger numbers
- **Gamma:** Requires bit-level operations (slower)

# PForDelta: Patched Frame of Reference

## Key Observation

In most posting lists, the vast majority of gaps fit in a small number of bits, but a few outliers are much larger

**PForDelta idea:**

1. Divide postings into blocks (e.g., 128 values)
2. Choose bit width $b$ that fits 90% of values in block
3. Store majority in $b$ bits each (packed tightly)
4. Store exceptions separately with positions

## Example

**Gaps:** [5, 3, 7, 512, 2, 4, 1, 6, 9, 3]

- Choose $b = 3$ bits (fits all except 512)

# PForDelta: Detailed Algorithm

**Encoding:**

```
Divide gaps into blocks of size B (e.g., 128)
for each block do
    Compute histogram of gap values
    Choose b such that ≥ 90% of values fit in b bits
    Pack values that fit into b-bit array
    For exceptions (values ≥ 2^b):
        Store position in block
        Store actual value separately
        Put special marker in main array
    Store: b, packed array, exception list
end for
```

**Decoding:**

1. Read $b$ (bit width)

# PForDelta: Performance

## Advantages

- **Excellent compression:** 5-10× typical
- **Very fast decompression:** SIMD-friendly (process 128 bits at once)
- **Random access:** Can decode individual blocks
- **Adaptive:** Bit width chosen per block

## Disadvantages

- **Complex implementation:** Bit packing, exception handling
- **Block-based:** Small overhead for block headers
- **Not universal:** Requires careful tuning

## Typical Performance

- Compression: 6-8× for typical web collections

# Compression Method Summary

| Method | Compression | Dec. Speed | Complexity | Use Case |
|---|---|---|---|---|
| None | 1× | Fastest | Trivial | Small indexes |
| VB | 3-4× | Fast | Simple | General purpose |
| Gamma | 4-5× | Medium | Moderate | Small gaps |
| PForDelta | 6-10× | Very Fast | Complex | Production |
| Simple-9/16 | 4-6× | Very Fast | Moderate | Production |

## Recommendations

- **Learning/prototyping:** Start with VB (easy to implement)
- **Production (small/medium):** VB (good enough, simple)
- **Production (large scale):** PForDelta or Simple-16 (best performance)
- **Research:** Experiment with adaptive schemes

# From Sparse to Dense Retrieval

## Sparse (Traditional)

**Representation:**

- Bag of words
- High-dimensional (vocab size)
- Mostly zeros (99%+ sparse)

**Index:** Inverted index
**Search:** AND/OR logic, BM25
**Limitation:** Vocabulary mismatch

## Dense (Neural)

**Representation:**

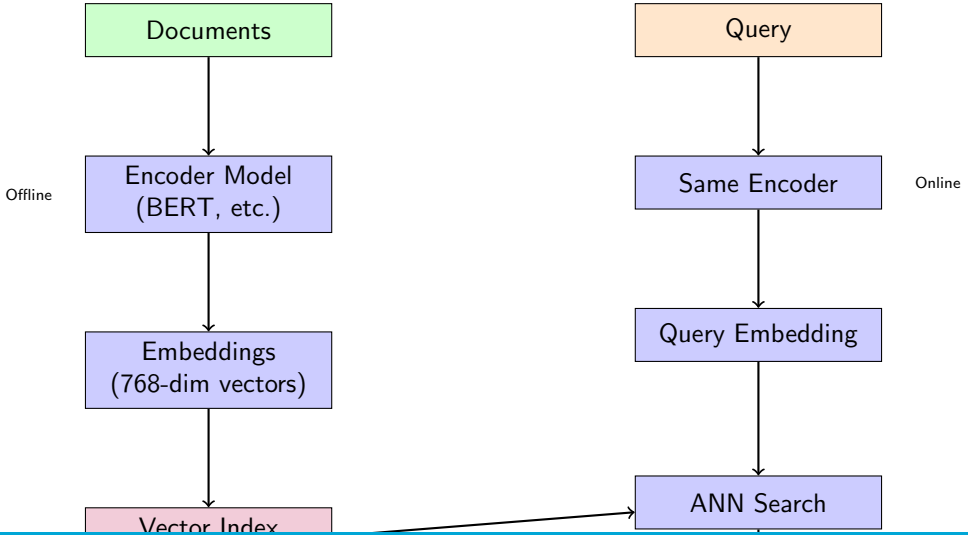
- Learned embeddings
- Low-dimensional (100-1000d)
- All non-zero (dense)

**Index:** Vector database
**Search:** Nearest neighbors
**Advantage:** Semantic matching

## Example

**Query:** "car problems"
**Sparse:** Misses document with "automobile issues"
**Dense:** Finds it! (similar embeddings)

# Dense Retrieval: The Pipeline



# The k-NN Search Problem

## Problem Definition

**Given:**

- Collection of $N$ document vectors: $\{\vec{d}_1, \vec{d}_2, \ldots, \vec{d}_N\}$
- Query vector: $\vec{q}$
- Distance/similarity function: $\text{sim}(\vec{q}, \vec{d})$ (e.g., cosine, dot product)

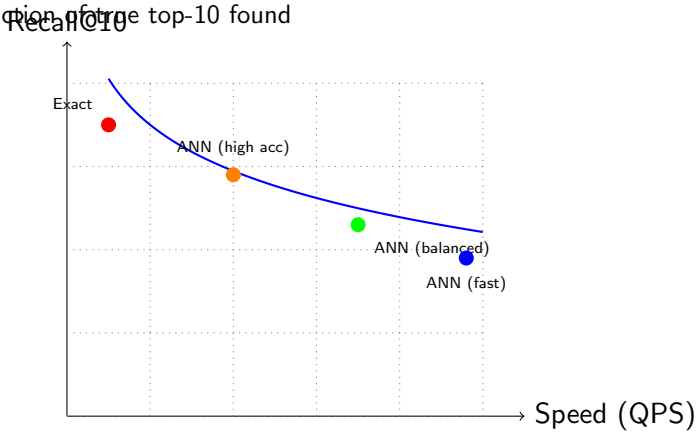
**Find:** $k$ documents with highest similarity to $\vec{q}$

## The Challenge

**Exact search:** Compare query to all $N$ vectors $\rightarrow O(N \cdot d)$ where $d =$ dimension
For $N = 10^9$ docs, $d = 768$:
$\rightarrow$ 768 billion operations per query
$\rightarrow$ Too slow! (¿1 second even on GPU)

# ANN: The Accuracy-Speed Trade-off

Recall@10: fraction of true top-10 found




More approximate → faster but less accurate

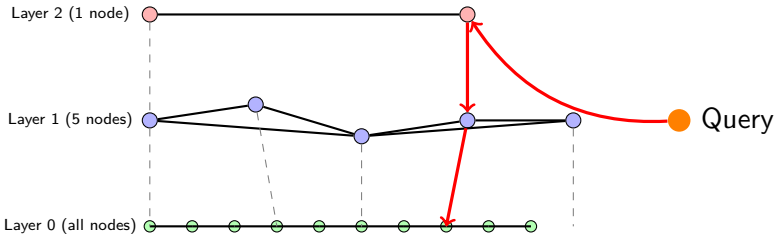
# Main ANN Approaches

1. **Graph-Based** (HNSW, NSG)
   - Build graph connecting similar vectors
   - Search by navigating graph
   - High accuracy, moderate memory
2. **Clustering/Inverted File (IVF, IMI)**
   - Cluster vectors (k-means)
   - Search only in nearest clusters
   - Fast, lower accuracy
3. **Product Quantization (PQ)**
   - Compress vectors aggressively
   - Approximate distances in compressed space
   - Very memory-efficient
4. **Tree-Based** (Annoy, KD-tree)
   - Partition space with trees
   - Search by tree traversal
   - Simple, good for moderate dimensions
5. **Hashing** (LSH)

# HNSW: Hierarchical Navigable Small World

## Core Idea

Build a multi-layer graph where upper layers provide "highways" for fast navigation to the target region




**Search:** Start at top, greedily navigate to nearest neighbor at each layer, descend to next layer, repeat

# HNSW: Construction

```
Input: Set of vectors V, parameter M (max connections per node)
Output: Multi-layer graph

Initialize empty graph
for each vector v⃗ in V do
    ℓ ← random layer (exponentially decaying probability)
    Insert v⃗ at layer ℓ and all layers below
    for each layer from top to ℓ do
        Find M nearest neighbors at this layer (using greedy search)
        Connect v⃗ to these neighbors
        if neighbor has ¿ M connections then
            Prune least useful edge
        end if
    end for
end for
```

# HNSW: Search

```
Input: Query q⃗, graph G, k (neighbors to find), ef (search width)
Output: Approximate k nearest neighbors

currLayer ← top layer
entryPoint ← entry node at top layer
candidates ← priority queue (min-heap by distance)
visited ← set of visited nodes
while currLayer ≥ 0 do
    Add entryPoint to candidates and visited
    while candidates not empty do
        nearest ← pop closest unvisited from candidates
        for each neighbor n of nearest at currLayer do
            if n not in visited then
                Add n to candidates and visited
                if distance(q⃗, n) ; distance to farthest in top-ef then
```

# HNSW: Properties

## Advantages

- ✓ Excellent recall (95-99% typical)
- ✓ Fast search: $O(\log N)$ average
- ✓ Supports updates (add/delete vectors)
- ✓ No training required
- ✓ Works well in high dimensions (100-1000d)

## Disadvantages

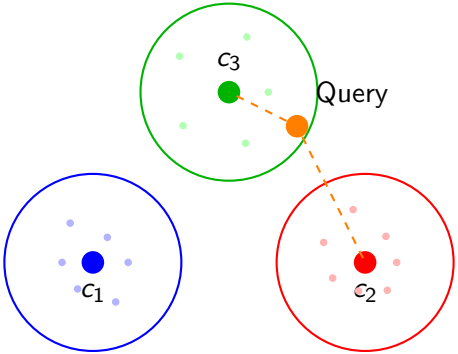
- ✗ High memory usage (graph structure: $\approx 2\times$ vector size)
- ✗ Slower to build than some alternatives
- ✗ Not as memory-efficient as quantization methods

**When to use:**

# IVF: Inverted File Index

**Core Idea**

Cluster vectors with k-means, then search only in nearest clusters




Search only in 1-2 nearest clusters

# IVF: Algorithm Details

**Index Construction:**

```
Run k-means on all vectors → get k cluster centers {c_1, ..., c_k}
Assign each vector to its nearest cluster center
Build inverted lists: cluster i → list of vectors in that cluster
```

**Search:**

```
Input: Query q⃗, n_probe (clusters to search), k (neighbors)
Compute distance from q⃗ to all cluster centers
Select n_probe nearest cluster centers
candidates ← []
for each selected cluster do
    Fetch all vectors in cluster
    Compute exact distances to q⃗
    Add to candidates
end for
Sort candidates by distance
return top-k candidates
```

**Key parameter:** $n_{probe}$ (trade recall for speed)

- $n_{probe} = 1$: Fast but low recall (search 1 cluster)
- $n_{probe} = 10$: Balanced (search 10 clusters)
- $n_{probe} = k$: Exact search (search all clusters)

# Product Quantization (PQ)

**Goal**

Compress vectors aggressively while preserving distance relationships

**Key idea:**

1. Split each $d$-dimensional vector into $m$ subvectors ($d/m$ dims each)
2. Quantize each subvector independently using k-means
3. Represent vector as $m$ cluster IDs (e.g., $m$ bytes)

**Example**

**Original:** 768-dim vector ($768 \times 4$ bytes $= 3072$ bytes)
**PQ compression:**

- Split into $m = 96$ subvectors of 8 dims each
- Quantize each subvector to $k = 256$ centroids (1 byte ID)
- Compressed: 96 bytes ($32\times$ compression!)

**Distance computation:**

- Pre-compute distances from query subvectors to all centroids
- Approximate distance = sum of subvector distances (lookup table)
- Very fast! (just $m$ additions)

# PQ: Detailed Algorithm

**Training:**

```
Split each vector into m subvectors of dimension d/m
for each subspace i = 1, ..., m do
    Run k-means on all subvectors in subspace i → get k centroids
    Store these centroids as codebook C_i
end for
```

**Encoding:**

```
Input: Vector v⃗
Split v⃗ into subvectors [v⃗_1, v⃗_2, ..., v⃗_m]
for each subvector v⃗_i do
    Find nearest centroid in C_i → get ID c_i
end for
Store vector as [c_1, c_2, ..., c_m] (m bytes if k=256)
```

# IVF + PQ: Best of Both Worlds

**IVFPQ: Combine IVF with Product Quantization**

1. Use IVF for coarse filtering (cluster-level search)
2. Use PQ for compression within clusters
3. Get both speed (IVF) and memory efficiency (PQ)

**Algorithm:**

1. Offline: Cluster vectors into $k$ clusters (IVF)
2. Offline: Within each cluster, train PQ codebooks
3. Offline: Encode all vectors with PQ
4. Online: Find $n_{probe}$ nearest clusters
5. Online: Search within those clusters using PQ distances

**Memory savings example:**

- Original: 1B vectors $\times$ 768 dims $\times$ 4 bytes $=$ 3TB
- IVFPQ: 1B vectors $\times$ 64 bytes (PQ) $=$ 64GB
- **50$\times$ compression!**

**Trade-off:** Lower recall than HNSW, but much more memory-efficient

# ANN Libraries Comparison

| Library | Method | Recall | Speed | Memory | Updates |
|---|---|---|---|---|---|
| HNSW (hnswlib) | Graph | Excellent | Fast | High | Yes |
| FAISS (IVF) | Clustering | Good | Very Fast | Medium | Limited |
| FAISS (IVFPQ) | Clustering+PQ | Good | Very Fast | Low | Limited |
| FAISS (HNSW) | Graph | Excellent | Fast | High | Yes |
| Annoy | Trees | Good | Fast | Medium | No |
| ScaNN | Learned quant. | Excellent | Very Fast | Low | Limited |
| Milvus | Multiple | Excellent | Fast | Varies | Yes |
| Weaviate | HNSW | Excellent | Fast | High | Yes |

## Choosing a Library

- **High recall needed:** HNSW (hnswlib, Weaviate)
- **Memory constrained:** FAISS IVFPQ, ScaNN
- **Maximum speed:** FAISS IVF (lower recall)
- **Distributed/production:** Milvus, Weaviate, Qdrant
- **Simple/prototyping:** Annoy, hnswlib

# Hybrid: Sparse + Dense

**Why Hybrid?**

Sparse and dense retrieval have complementary strengths

**Sparse (BM25) strengths:**

- Exact term matching
- Rare/specific terms
- Named entities
- Acronyms, IDs
- Explainable

**Dense strengths:**

- Semantic matching
- Synonyms
- Paraphrases
- Conceptual similarity
- Cross-lingual

**Hybrid approaches:**

1. **Parallel retrieval + fusion**
   - Run sparse and dense in parallel
   - Merge results (e.g., reciprocal rank fusion)

# Key Takeaways: Posting Lists

1. **Posting list structure**
   - Core: (docID, frequency, positions)
   - Sorted by docID for merge operations
   - Variable length (power-law distribution)
2. **Skip lists**
   - Add "highways" for faster intersection
   - Place every $\sqrt{n}$ elements typically
   - Trade space for speed

# Key Takeaways: Query Processing

1. **DAAT vs. TAAT**
   - DAAT: Process one document at a time (modern systems)
   - TAAT: Process one term at a time (batch scoring)
2. **Optimizations**
   - MaxScore: Skip low-impact terms
   - WAND: Dynamic pruning with upper bounds
   - Impact-ordered indexes: Early termination

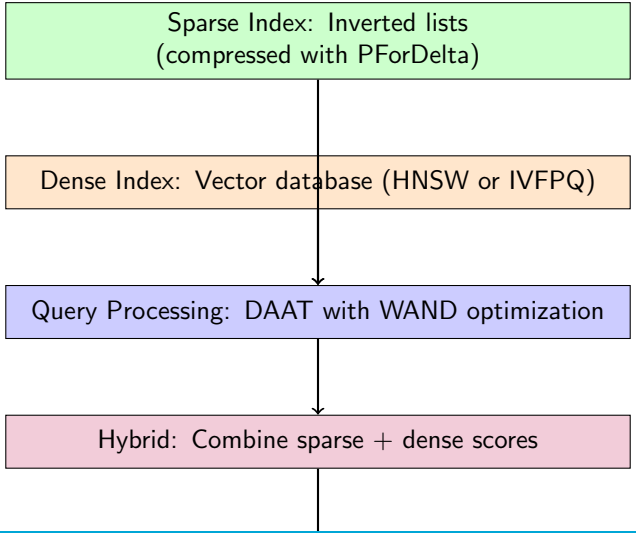
# Key Takeaways: Compression

1. **Gap encoding is essential**
   - Store gaps instead of absolute docIDs
   - Most gaps are small (compress well)
2. **Compression methods**
   - VB: Simple, fast, 3-4$\times$ compression
   - Gamma: Better for small numbers
   - PForDelta: Best performance, 6-10$\times$ compression
3. **Why it matters**
   - Smaller index $\rightarrow$ fits in RAM
   - Faster I/O $\rightarrow$ better query latency
   - Disk is slow, CPU is fast

# Key Takeaways: ANN Indexing

1. **Dense retrieval requires ANN**
   - Exact search too slow for billion-scale
   - Trade recall for speed (90-99% recall acceptable)
2. **Main approaches**
   - HNSW: Graph-based, excellent recall
   - IVF: Clustering, fast but needs more probes
   - PQ: Aggressive compression, memory-efficient
   - IVFPQ: Combines IVF + PQ (production standard)
3. **Hybrid is often best**
   - Sparse for exact matching
   - Dense for semantic matching
   - Combine for best of both worlds

# The Complete Modern IR Stack



# Questions?

Thank you!

Questions?

**Next: Lecture 3 — Ranking Models**

We'll use these indexes to implement BM25, language models, and query expansion with RM3!